📄 ส่วนที่ 1: Service Design & Architecture Decisions (20 คะแนน)

ในการเพิ่มฟีเจอร์ "Team Chat" เข้าไปใน Task Board System จำเป็นต้องมีการออกแบบสถาปัตยกรรมที่รองรับปริมาณผู้ใช้งานและลักษณะการทำงานที่เปลี่ยนแปลงไป โดยมีรายละเอียดการตัดสินใจ (Architecture Decisions) ดังนี้:

1. ควรสร้าง Service ใหม่ (Chat Service) หรือเพิ่มใน Task Service?

✅ **การตัดสินใจ: สร้าง Service ใหม่ (Chat Service) แยกออกมาเป็น Microservice อย่างเด็ดขาด**

เหตุผลเชิงสถาปัตยกรรม (Architectural Justification):

- **Separation of Workloads:** Task Service (Lab 4) มีรูปแบบการทำงานเป็นแบบ Request-Response (Stateless) เน้นการทำ CRUD Operations กับฐานข้อมูลเป็นหลัก ในขณะที่ Chat Service ต้องการการเชื่อมต่อแบบ Persistent (Stateful) เพื่อเปิดรับส่งข้อมูลแบบ Real-time ตลอดเวลา การรวมกันจะทำให้การจัดการทรัพยากร (CPU/Memory/Connections) ทำได้ยาก
- **Independent Scalability:** ฟีเจอร์แชทมีอัตราการเกิด I/O (Input/Output) ที่สูงกว่าการจัดการ Task Board หลายเท่า การแยก Service จะทำให้เราสามารถ Scale out เฉพาะ Chat Service (ดังที่แสดงในแผนภาพว่ามี Instance 1 และ 2) ได้โดยไม่เปลืองทรัพยากรของ Task Service
- **Fault Isolation (การจำกัดความเสียหาย):** หากระบบแชทล่มจากปริมาณข้อความที่มหาศาล (Spike traffic) ผู้ใช้งานก็ยังสามารถจัดการสร้าง-ลบ Task ใน Board ได้ตามปกติผ่าน Task Service

เกณฑ์เปรียบเทียบ	หากเพิ่มใน Task Service (Monolith)	แยกสร้าง Chat Service ใหม่ (Microservices)
การใช้ทรัพยากร	แย่ง Memory/Thread กัน อาจทำให้ระบบรวมช้า	แยกการคำนวณชัดเจน จัดการ Resource ได้ดีกว่า
การขยายระบบ (Scale)	ต้อง Scale ทั้งก้อน เปลืองค่า Server	Scale เฉพาะส่วน Chat ที่มีการใช้งานหนักได้
ความเสถียร (Resilience)	แชทล่ม = ระบบ Task Board ล่มไปด้วย	แชทล่ม = แจ้งเตือนแชทไม่พร้อมใช้งาน แต่ Task ยังทำงานได้

2. ถ้าสร้าง Service ใหม่ ⟶ Technology Stack อะไร? เพราะอะไร?

✅ **การตัดสินใจ:** ใช้ **Node.js** ร่วมกับ **Socket.io** (และ Express สำหรับ REST API)

เหตุผลเชิงเทคโนโลยี (Technical Rationale):

1. **Event-Driven & Non-blocking I/O:** สถาปัตยกรรมของ Node.js ถูกออกแบบมาให้รับมือกับ Concurrent Connections จำนวนหลายหมื่นคอนเนคชันพร้อมกันได้ดีเยี่ยมด้วย Thread เดียว (Event Loop) ทำให้ประหยัด RAM คุ้มค่าที่สุดสำหรับแอปพลิเคชันแชท
2. **Built-in Room Management:** ไลบรารี Socket.io มีฟีเจอร์ "Rooms" และ "Namespaces" ซึ่งตรงกับ Requirement ของ "Task Board" พอดี (อ้างอิงจากแผนภาพ Event Flow ที่มี Event: join_room {boardId}) ทำให้เราสามารถ Broadcast ข้อความเฉพาะใน Board นั้นๆ ได้ทันทีโดยไม่ต้องเขียนตรรกะกรองข้อมูลเอง
3. **Fallback Mechanism:** Socket.io มีระบบอัตโนมัติที่จะปรับลดไปใช้ HTTP Long-Polling หากเครือข่ายของ Client ไม่รองรับ WebSocket

องค์ประกอบ	**เทคโนโลยีที่เลือก**	**หน้าที่และเหตุผล**
Runtime Engine	**Node.js**	รองรับ I/O-bound tasks สูง จัดการ WebSocket ได้มีประสิทธิภาพ
Real-time Library	**Socket.io**	จัดการ Connection, มีระบบ Room สำหรับแยกบอร์ด, มี Auto-reconnect

องค์ประกอบ	เทคโนโลยีที่เลือก	หน้าที่และเหตุผล
API Framework	**Express.js**	จัดการ REST API (เช่น การ Upload ไฟล์ และ ดึงประวัติแชทเก่า)

3. Database ควรใช้อะไร? (SQL/NoSQL/Cache) เพราะอะไร?

✅ **การตัดสินใจ:** ใช้สถาปัตยกรรม **Polyglot Persistence** (ผสมผสาน Database หลายประเภทตามความเหมาะสมของข้อมูล) โดยอิงตาม Architecture Diagram ดังนี้:

เหตุผลและการเลือกใช้: การแชทต้องการความเร็วในการเขียน (High Write Throughput) ในขณะที่ข้อมูลมีลักษณะเป็นแบบ Time-series และมีโครงสร้างยืดหยุ่น (บางข้อความเป็น Text, บางข้อความเป็นรูปภาพ) การใช้ Relational DB (SQL) แบบเดิมอาจเกิดปัญหา Lock Table หรือขยายระบบยาก

ประเภทข้อมูล (Data Type)	**เทคโนโลยีที่เลือก (Database)**	**เหตุผลทางสถาปัตยกรรม (Why?)**
ข้อมูลแชท (Chat History)	**NoSQL (MongoDB)**	รองรับการเขียนข้อมูลมหาศาล (High Write) โครงสร้าง Document (BSON) ยืดหยุ่นรองรับรูปแบบข้อความที่หลากหลาย สเกลแบบ Horizontal (Sharding) ได้ง่าย
การซิงค์ข้อมูล / สถานะ	**Cache / Message Broker (Redis)**	จำเป็นอย่างยิ่งเมื่อมี Chat Service มากกว่า 1 Instance (Pub/Sub Broker) เพื่อให้เซิร์ฟเวอร์ทุกตัวสามารถส่งข้อความข้ามกันได้ และใช้เก็บ Session ชั่วคราว (Online status) ความเร็วระดับ In-memory

ประเภทข้อมูล (Data Type)	**เทคโนโลยีที่เลือก (Database)**	**เหตุผลทางสถาปัตยกรรม (Why?)**
ไฟล์แนบ / รูปภาพ	**Object Storage (AWS S3)**	Database ไม่ควรใช้เก็บไฟล์ Binary การใช้ S3 ช่วยแยกภาระการเก็บไฟล์รูปภาพ/เอกสาร และรองรับการทำ CDN เพื่อให้ดาวน์โหลดได้รวดเร็ว
(ข้อมูลระบบเดิม Lab 4)	*(SQL - SQLite / PostgreSQL)*	*(เก็บข้อมูลโครงสร้าง Board, Task, User ที่ต้องการความสัมพันธ์และ ACID Properties สูง)*

4. Real-time ใช้ WebSocket หรือ Polling? Trade-offs?

✅ **การตัดสินใจ:** เลือกใช้ **WebSocket (WSS)** เป็นหลักในการรับส่งข้อความ

เหตุผลเชิงประสิทธิภาพ: WebSocket เป็นการสร้างช่องทางการสื่อสารแบบ **Full-duplex TCP Connection** หมายความว่าเมื่อ Handshake สำเร็จครั้งแรก ท่อส่งข้อมูลจะเปิดค้างไว้ เซิร์ฟเวอร์สามารถผลัก (Push) ข้อมูลหาผู้ใช้ได้ทันทีโดยที่ผู้ใช้ไม่ต้องส่ง Request มาร้องขอ ตอบโจทย์ Real-time Chat ที่ต้องการ Latency ต่ำที่สุด

การวิเคราะห์ข้อดี-ข้อเสีย (Trade-offs Analysis):

คุณสมบัติ	**WebSocket (ทางเลือกที่ใช้)**	**HTTP Polling (ทางเลือกที่ตัดทิ้ง)**
ทิศทางข้อมูล	**2 ทาง (Full-duplex)** ส่ง-รับได้อิสระตลอดเวลา	**1 ทาง (Half-duplex)** Client ต้องเป็นฝ่ายถาม Server ตลอด
Network Overhead	**ต่ำมาก** (ใช้ Header แค่ไม่กี่ Byte หลัง Handshake)	**สูง** (ต้องส่ง HTTP Headers ทุกๆ ครั้งที่ยิง Request)
ความหน่วง (Latency)	**ต่ำสุด (Real-time แท้จริง)** ข้อความเด้งทันที	**สูง (Near Real-time)** ขึ้นอยู่กับความถี่ในการตั้งรอบดึงข้อมูล

คุณสมบัติ	WebSocket (ทางเลือกที่ใช้)	HTTP Polling (ทางเลือกที่ตัดทิ้ง)
ความยากในการขยายระบบ	❌ **ยาก (Stateful)** ต้องใช้กลไก Sticky Sessions บน Load Balancer และต้องมี Redis (Pub/Sub) เข้ามาช่วยซิงค์ข้าม Instance	✅ **ง่าย (Stateless)** Load Balancer โยนไปเข้า Instance ไหนก็ได้

บทสรุป Trade-off: แม้ WebSocket จะทำให้สถาปัตยกรรมระบบมีความซับซ้อนเพิ่มขึ้น (ต้องเพิ่ม Redis และตั้งค่า Load Balancer แบบพิเศษ) แต่เพื่อแลกกับประสบการณ์ผู้ใช้ (UX) ที่ลื่นไหล ไม่หน่วง และการประหยัด Bandwidth ของเซิร์ฟเวอร์ในระยะยาว จึงเป็นทางเลือกที่คุ้มค่าและถูกต้องตามมาตรฐานอุตสาหกรรม (Industry Standard) สำหรับฟีเจอร์ระบบแชท